NumPy 2.0.0 Release Notes

> **Note**
>
> The release of 2.0 is in progress and the current release overview and highlights are still in a draft state. However, the highlights should already list the most significant changes detailed in the full notes below, and those full notes should be complete (if not copy-edited well enough yet).

NumPy 2.0.0 is the first major release since 2006. It is the result of 11 months of development since the last feature release and is the work of 212 contributors spread over 1078 pull requests. It contains a large number of exciting new features as well as changes to both the Python and C APIs.

This major release includes breaking changes that could not happen in a regular minor (feature) release - including an ABI break, changes to type promotion rules, and API changes which may not have been emitting deprecation warnings in 1.26.x. Key documents related to how to adapt to changes in NumPy 2.0, in addition to these release notes, include:

- The NumPy 2.0 migration guide
- The NumPy 2.0-specific advice in For downstream package authors

Highlights

Highlights of this release include:

- New features:
  - A new variable-length string dtype, `StringDType` and a new `numpy.strings` namespace with performant ufuncs for string operations,
  - Support for `float32` and `longdouble` in all `numpy.fft` functions,
  - Support for the array API standard in the main `numpy` namespace.
- Performance improvements:
  - Sorting functions (`sort`, `argsort`, `partition`, `argpartition`) have been accelerated through the use of the Intel x86-simd-sort and Google Highway

- The building from source documentation was completely rewritten,

Furthermore there are many changes to NumPy internals, including continuing to migrate code from C to C++, that will make it easier to improve and maintain NumPy in the future.

The “no free lunch” theorem dictates that there is a price to pay for all these API and behavior improvements and better future extensibility. This price is:

1. Backwards compatibility. There are a significant number of breaking changes to both the Python and C APIs. In the majority of cases, there are clear error messages that will inform the user how to adapt their code. However, there are also changes in behavior for which it was not possible to give such an error message - these cases are all covered in the Deprecation and Compatibility sections below, and in the NumPy 2.0 migration guide.

   Note that there is a `ruff` mode to auto-fix many things in Python code.

2. Breaking changes to the NumPy ABI. As a result, binaries of packages that use the NumPy C API and were built against a NumPy 1.xx release will not work with NumPy 2.0. On import, such packages will see an `ImportError` with a message about binary incompatibility.

   It is possible to build binaries against NumPy 2.0 that will work at runtime with both NumPy 2.0 and 1.x. See NumPy 2.0-specific advice for more details.

   All downstream packages that depend on the NumPy ABI are advised to do a new release built against NumPy 2.0 and verify that that release works with both 2.0 and 1.26 - ideally in the period between 2.0.0rc1 (which will be ABI-stable) and the final 2.0.0 release to avoid problems for their users.

The Python versions supported by this release are 3.9-3.12.

NumPy 2.0 Python API removals

- `np.geterrobj`, `np.seterrobj` and the related ufunc keyword argument `extobj=` have been removed. The preferred replacement for all of these is using the context manager `with np.errstate():`.

  (gh-23922)

- `np.cast` has been removed. The literal replacement for `np.cast[dtype](arg)` is `np.asarray(arg, dtype=dtype)`.
- `np.source` has been removed. The preferred replacement is `inspect.getsource`.
- `np.lookfor` has been removed.
  (gh-24144)

- `numpy.who` has been removed. As an alternative for the removed functionality, one can use a variable explorer that is available in IDEs such as Spyder or Jupyter Notebook.
  (gh-24321)

- Warnings and exceptions present in **`numpy.exceptions`** (e.g, **`ComplexWarning`**, **`VisibleDeprecationWarning`**) are no longer exposed in the main namespace.
- Multiple niche enums, expired members and functions have been removed from the main namespace, such as: `ERR_*`, `SHIFT_*`, `np.fastCopyAndTranspose`, `np.kernel_version`, `np.numarray`, `np.oldnumeric` and `np.set_numeric_ops`.
  (gh-24316)

- Replaced `from ... import *` in the `numpy/__init__.py` with explicit imports. As a result, these main namespace members got removed: `np.FLOATING_POINT_SUPPORT`, `np.FPE_*`, `np.NINF`, `np.PINF`, `np.NZERO`, `np.PZERO`, `np.CLIP`, `np.WRAP`, `np.WRAP`, `np.RAISE`, `np.BUFSIZE`, `np.UFUNC_BUFSIZE_DEFAULT`, `np.UFUNC_PYVALS_NAME`, `np.ALLOW_THREADS`, `np.MAXDIMS`, `np.MAY_SHARE_EXACT`, `np.MAY_SHARE_BOUNDS`, `add_newdoc`, `np.add_docstring` and `np.add_newdoc_ufunc`.
  (gh-24357)

- Alias `np.float_` has been removed. Use `np.float64` instead.
- Alias `np.complex_` has been removed. Use `np.complex128` instead.
- Alias `np.longfloat` has been removed. Use `np.longdouble` instead.
- Alias `np.singlecomplex` has been removed. Use `np.complex64` instead.
- Alias `np.cfloat` has been removed. Use `np.complex128` instead.
- Alias `np.longcomplex` has been removed. Use `np.clongdouble` instead.
- Alias `np.clongfloat` has been removed. Use `np.clongdouble` instead.
- Alias `np.string_` has been removed. Use `np.bytes_` instead.
- Alias `np.unicode_` has been removed. Use `np.str_` instead.

- Alias `np.Inf` has been removed. Use `np.inf` instead.
- Alias `np.Infinity` has been removed. Use `np.inf` instead.
- Alias `np.NaN` has been removed. Use `np.nan` instead.
- Alias `np.infty` has been removed. Use `np.inf` instead.
- Alias `np.mat` has been removed. Use `np.asmatrix` instead.
- `np.issubclass_` has been removed. Use the `issubclass` builtin instead.
- `np.asfarray` has been removed. Use `np.asarray` with a proper dtype instead.
- `np.set_string_function` has been removed. Use `np.set_printoptions` instead with a formatter for custom printing of NumPy objects.
- `np.tracemalloc_domain` is now only available from `np.lib`.
- `np.recfromcsv` and `recfromtxt` are now only available from `np.lib.npyio`.
- `np.issctype`, `np.maximum_sctype`, `np.obj2sctype`, `np.sctype2char`, `np.sctypes`, `np.issubsctype` were all removed from the main namespace without replacement, as they where niche members.
- Deprecated `np.deprecate` and `np.deprecate_with_doc` has been removed from the main namespace. Use `DeprecationWarning` instead.
- Deprecated `np.safe_eval` has been removed from the main namespace. Use `ast.literal_eval` instead.
  (gh-24376)

- `np.find_common_type` has been removed. Use `numpy.promote_types` or `numpy.result_type` instead. To achieve semantics for the `scalar_types` argument, use `numpy.result_type` and pass `0`, `0.0`, or `0j` as a Python scalar instead.
- `np.round_` has been removed. Use `np.round` instead.
- `np.nbytes` has been removed. Use `np.dtype(<dtype>).itemsize` instead.
  (gh-24477)

- `np.compare_chararrays` has been removed from the main namespace. Use `np.char.compare_chararrays` instead.
- The `charrarray` in the main namespace has been deprecated. It can be imported without a deprecation warning from `np.char.chararray` for now, but we are planning to fully deprecate and remove `chararray` in the future.

Constants

NumPy includes several constants:

numpy.e

Euler’s constant, base of natural logarithms, Napier’s constant.

```
e = 2.71828182845904523536028747135266249775724709369995...
```

See Also

exp : Exponential function log : Natural logarithm

References

https://en.wikipedia.org/wiki/E_%28mathematical_constant%29

numpy.euler_gamma

```
y = 0.57721566490153286060651209008240243104215...
```

References

https://en.wikipedia.org/wiki/Euler-Mascheroni_constant

numpy.inf

IEEE 754 floating point representation of (positive) infinity.

Returns

y : *float*

A floating point representation of positive infinity.

See Also

isinf : Shows which elements are positive or negative infinity

isposinf : Shows which elements are positive infinity

isneginf : Shows which elements are negative infinity

isnan : Shows which elements are Not a Number

isfinite : Shows which elements are finite (not one of Not a Number, positive infinity and negative infinity)

Notes

NumPy uses the IEEE Standard for Binary Floating-Point for Arithmetic (IEEE 754). This means that Not a Number is not equivalent to infinity. Also that positive infinity is not equivalent to negative infinity. But infinity is equivalent to positive infinity.

Examples

```
>>> np.inf
inf
>>> np.array([1]) / 0.
array([inf])
```

numpy.nan

IEEE 754 floating point representation of Not a Number (NaN).

Returns

y : A floating point representation of Not a Number.

See Also

isnan : Shows which elements are Not a Number.

isfinite : Shows which elements are finite (not one of Not a Number, positive infinity and negative infinity)